

Project Report: Text-Based RPG Engine

A Demonstration of Core Object-Oriented Programming Principles using a text based simple game

Course Name: Object Oriented Programming

Submitted By: (Group 5) Bittu Shah 24103014 , Moramba Sougrakpam 24103041 , Rohen Khoisnam 24103052 , Sothang Meitei 24103057

Date: 22nd April 2026

Abstract

This project involves the design and implementation of a text-based, turn-based Role-Playing Game (RPG) engine written in C++20. The primary objective is to serve as a comprehensive sandbox demonstrating the core pillars of Object-Oriented Programming (OOP): Encapsulation, Inheritance, Polymorphism, and Abstraction. Beyond demonstrating the basics of OOP concepts, the project involves various software engineering paradigms, including Aggregation, Resource Acquisition Is Initialization (RAII) memory management via smart pointers, the User-Target design pattern, and highly efficient algorithmic data sweeping. By strictly separating backend math logic from the terminal presentation layer, the application ensures a robust, extensible, and memory-safe architecture.

1. Introduction

Project Objective

Here we try to demonstrate the core principles of Object Oriented Programming using this project.

Scope of the Project

The project is a turn based combat arena game where there is only one player and various enemies, the player and all the enemies take turn to attack each other. It does not involve the use of graphics library like SDL and solely rely on a text based terminal output to show the in game events. The project then has various classes and files to represent the type of entities and items in the game and runtime polymorphic behaviours. It includes features like that of the inventory management , dynamic damage calculations and a ASCII - based visual feedback mechanism.

Technology Stack

This project is entirely written in the standard c++ 20 without relying on any other third party libraries , all the libraries are either user defined or within the STL of the c++ 20 standard

Language: C++ (Standard: C++20)

Key Libraries: <memory> (for RAII), <vector>, <chrono> & <thread> (for animation pacing), <algorithm> (for container mutations).

2. System Architecture & Design

The Game Loop Architecture

The core engine runs a while loop for as long as the boolean flag of game is running is still true each while loop iteration is treated as a frame. The game has a Battle manager class that stores the main player and the enemies. For each iteration the following events happen in sequential order

First The Player Phase(as the player moves first in the turn based game):

Here the user input gets handled and it triggers polymorphic behaviours of the items that are stored or held by the player.

The Deletion of the Dead or inactive Enemies Phase:

A linear time algorithm method of the Battle manager is called to delete the dead enemy entity from the array of enemies stored by the manager.

Attack by enemies phase:

There is a linear time iteration through the surviving entities and their AI logic is called for each of them that calculates damage and triggers the respective animations of each of the enemies.

Conceptual Class Hierarchy

The software utilizes a dual-hierarchy system:

- **The Entity Hierarchy:** A base entity class from which player and enemy inherit.
- **The Item Hierarchy:** A purely abstract item base class from which specialized items like weapon and potion are then derived.

The User-Target Interface Pattern

To prevent tight coupling between items and entities, there is implementation of the user-target pattern in the virtual use method in the abstract base class of the item class here , the user of the item and the target entity of the item is passed into the method of the item instance and the item makes itself be used by the user entity and it does damage to the target entity.

3. Core OOP Implementation

1 Inheritance (The "Is-A" Relationship)

Inheritance is implemented when the more abstract classes get inherited by the more specialized classes, thus, inheriting their properties. This is seen in the entity and the player, enemy relationship and the item to weapon position relationship. This makes sure that the common properties of the player and the enemy being an entity is shared and capitalized.

2 Polymorphism (Dynamic Dispatch)

Polymorphism is demonstrated through the resolution of overridden functions at runtime. The player inventory holds a vector of abstract item pointers. With the user input the player at runtime selects an item from this inventory array and then uses this item, but depending on what type of item it is say a weapon or potion its respective use method of the item is called, this is made possible as the item class is a pure abstract class that is inherited by the derived class of the weapon or the potion class.

3 Encapsulation

Internal state management is strictly hidden from the global scope. Class variables such as `m_health`, `m_damage`, and `m_items` are locked behind protected or private access modifiers. There are getters and setters for each of the internal member variables thus, protecting the variables from accidental outside mutation. Thus demonstrating the use of the encapsulation principle.

4 Abstraction

Abstraction is achieved by hiding complex backend logic behind simplified method calls. In this project there are several methods in many of the classes that have a simple interface but it has a lot of complex internal mechanisms hidden from the user. Like that of the use method in the item class this method implements some complex mechanisms like that of the mutation of the variables of the other entities that remains hidden from the user. and almost all of the animation class is a form of abstraction as it has implementation that the user does not know, the user simply must satisfy some preconditions to use this method without ever having the knowledge of the inner working of the methods.

4. Other Software Engineering Concepts

Aggregation and Composition

The BattleManager class highlights Aggregation (a "Has-A" relationship). The manager is *not* an entity itself, nor does it strictly manufacture them (like a Factory pattern). Instead, it manages the lifecycle and interaction rules of a collection of entities, thus it has an array of entities that are present in the battle context (`std::vector<std::shared_ptr<entity>> enemies`). This is to make

sure that this is a decoupled logic system.

Modern Memory Management (RAII)

Here the project uses modern c++ idioms and practices for better and safer code that has clearer intent and is more efficient to understand and is safer in general. It makes use of `std::shared_ptr` by the owner class or the owner object to hold instances of objects so as to not make unnecessary copies. It makes use of the RAII concept where the owner is responsible for deallocation of its own memory. And it makes use of several containers and functions defined in the c++ 20 STL.

Algorithmically efficient($O(N)$ Sweeping)

To manage the removal of dead entities, the engine avoids computationally heavy, manual nested loops. Instead, it utilizes the C++20 `std::erase_if` algorithm in tandem with a stateless lambda expression. This allows the program to iterate through the active enemies array exactly once ($O(N)$ time complexity), efficiently shifting surviving elements and resizing the vector without performance degradation.

Challenges and Solutions

Terminal Frame-Wiping and Pacing

Challenge: there were several challenges to the implementation of the text animation output to the screen, it was difficult to make sure that the animations were timed correctly and that the input and the animations matched correctly and what type of animation felt natural to the user or the game player, after several iterations the most natural looking method was implemented.

Solution: the logic was refactored and improved after looking for bugs and errors at each point in each iteration, the implementation and placement of the `waitNextMove()` function was improved at each point, finally after human testing after the best or better UI was obtained it was then implemented.

Conclusion and Future Scope

This project successfully demonstrates a highly structured, memory-safe text RPG engine and the core principles of OOP in the following manner

1) Inheritance is demonstrated

in the part where

the enemy and the player class inherit from the class entity so

enemy is an entity and player is an entity

item is inherited by the classes potion and the class weapon

so that potion is an item and weapon is an item

2) polymorphism is demonstrated when the purely abstract class of the item class is inherited and then

its internal member functions are overridden by the derived classes of potions and weapon to define their class specific

implementations of the function use, which depending on the context of the item of what item it is the will differ the

so that when the method use is called on the item if the item is a potion then the potion implementation is called

else if the item is a weapon then the weapon implementation of the use method is called

3) Encapsulation

here in the program all the classes have getters and setters and the internal member variables are hidden from the user

thus demonstrating the idea of Encapsulation

4) Abstraction

Abstraction is demonstrated by reducing complex underlying combat mathematics into simple interfaces.

For example, when a weapon is used, the main loop does not calculate the damage multipliers or health reductions.

It simply calls item->use(this, target), and the complexity of the stat calculation is

abstracted away

behind that single method call.

5) Aggregation

BattleManager class demonstrates Aggregation.

A BattleManager is not an entity, but it has a collection of entities
(std::vector<std::shared_ptr<entity>> enemies)

Algorithmic Memory Management (BattleManager.h)

```
void clearDeadEnemies() {  
    // O(N) single-pass memory sweep leveraging C++20 algorithm  
    std::erase_if(enemies, [](const std::shared_ptr<entity>& e) {  
        if (!e->isAlive()) {  
            std::cout << e->getName() << " has been defeated!\n";  
            return true;  
        }  
        return false;  
    });  
}
```

The User-Target Polymorphic Interface (weapon.h)

```
void use(entity* user, entity* target) override {  
    int totalDamage = m_baseDamage + user->getDamage();  
    target->set_health(target->getHealth() - totalDamage);  
    attack_animation();  
  
    std::cout << "\n" << user->getName() << " attacks " << target->getName()  
        << " for " << totalDamage << " damage!\n";  
  
    if(target->getHealth() <= 0){  
        target->set_isAlive(false);  
    }  
}
```